A hash-like object representing the current session data. We describe this in *Rails Sessions*.

In addition, a `logger` is available throughout Action Pack.

Responding to the User

Part of the controller's job is to respond to the user, which is done in four ways:

- The most common way is to render a template. In terms of the MVC paradigm, the template is the view, taking information provided by the controller and using it to generate a response to the browser.
- The controller can return a string directly to the browser without invoking a view. This is fairly rare but can be used to send error notifications.
- The controller can return nothing to the browser. This is sometimes used when responding to an Ajax request. In all cases, however, the controller returns a set of HTTP headers because some kind of response is expected.
- The controller can send other data to the client (something other than HTML). This is typically a download of some kind (perhaps a PDF document or a file's contents).

A controller always responds to the user exactly one time per request. This means you should have just one call to a `render`, `redirect_to`, or `send_`*xxx* method in the processing of any request. (A `DoubleRenderError` exception is thrown on the second render.)

Because the controller must respond exactly once, it checks to see whether a response has been generated just before it finishes handling a request. If not, the controller looks for a template named after the controller and action and automatically renders it. This is the most common way that rendering takes place. You may have noticed that in most of the actions in our shopping cart tutorial we never explicitly rendered anything. Instead, our action methods set up the context for the view and return. The controller notices that no rendering has taken place and automatically invokes the appropriate template.